

RELATÓRIO 3 - LABORATÓRIO 10/09

Análise de Algoritmos Aplicada ao Cadastro de Bolsistas da CAPES

Disciplina: Análise de Algoritmos - Ciência da Computação - UCB

Grupo 5

Repositório: <https://github.com/branchescunha/algorithm-analysis/tree/main/report-03-scholarship-registration-analysis>

1. Introdução

O exercício trata da importação de registros de bolsistas armazenados em um arquivo CSV para outro cadastro, garantindo que nenhum nome já existente seja inserido novamente. A redundância é definida exclusivamente pelo campo Nome. Além de implementar a solução em C, o trabalho exige analisar formalmente quantas comparações entre nomes são realizadas e verificar, com dados experimentais, se a implementação segue o modelo matemático previsto.

2. Proposta de algoritmo e solução implementada

A solução utiliza busca sequencial no arquivo de destino. Para cada linha da origem, o programa extrai o Nome, percorre os registros já existentes no destino e compara os nomes um a um. Se encontrar igualdade, informa o nome redundante e não copia a linha; caso contrário, copia a linha completa, preservando Modalidade, Nível e Agência.

2.1 Pseudocódigo

```
comparacoes <- 0
inseridos <- 0
para cada registro da origem:
    nome <- extrair campo Nome
    repetido <- falso
    para cada registro já existente no destino:
        comparacoes <- comparacoes + 1
        se nome == nome_do_destino:
            repetido <- verdadeiro
            interromper a busca
    se repetido:
        exibir erro e continuar
    senão:
        copiar linha completa para o destino
        inseridos <- inseridos + 1
gravar inseridos e comparacoes no CSV de estatísticas
```

2.2 O que foi feito, como foi feito e por quê

O QUE	COMO	POR QUÊ
Busca de redundância	O destino é percorrido linha a linha; o Nome extraído da origem é comparado com cada Nome já cadastrado usando strcmp.	A busca sequencial foi exigida no enunciado e permite relacionar diretamente a execução ao número de comparações analisado.
Critério de igualdade	Somente o campo Nome é extraído antes da primeira vírgula e usado na comparação.	O enunciado define que registros com o mesmo Nome são redundantes mesmo que Modalidade, Nível ou Agência sejam diferentes.
Contador de comparações	O contador é incrementado exatamente uma vez antes de cada comparação Nome×Nome.	Assim a métrica observada corresponde exatamente à grandeza definida pelo professor, sem depender de tempo de CPU.
Cópia do registro	Quando o nome não é encontrado, a linha original inteira é anexada ao destino.	Isso preserva todos os campos e evita reconstruir ou alterar os dados recebidos.
Tratamento de repetição	Ao encontrar igualdade, a busca termina para aquele nome, o erro é exibido e o próximo registro é processado.	Cumprir a regra de não inserir duplicados sem interromper toda a importação.

2.3 Estruturas de dados utilizadas

O programa usa FILE* para abrir e percorrer os arquivos, vetores de char para armazenar cada linha e o Nome extraído, e variáveis long long para os contadores de inserções e comparações. O tipo long long foi escolhido porque o número de comparações cresce quadraticamente e pode ultrapassar a faixa de um int para entradas grandes. Não foi utilizada tabela hash, árvore ou índice auxiliar, pois isso mudaria o método de busca solicitado e, consequentemente, o modelo de comparações.

3. Análise teórica

Considere n novos bolsistas, todos distintos e inexistentes no destino, e Q registros já cadastrados antes do início. O primeiro novo nome é comparado com Q registros; depois de ser inserido, o destino passa a ter $Q+1$ registros. O segundo exige $Q+1$ comparações, e assim sucessivamente até o n -ésimo, que exige $Q+n-1$.

$$C(n, Q) = Q + (Q+1) + (Q+2) + \dots + (Q+n-1)$$

Separando os Q repetidos do somatório dos incrementos:

$$C(n, Q) = nQ + [0 + 1 + 2 + \dots + (n-1)]$$

$$C(n, Q) = nQ + n(n-1)/2$$

3.1 Quantidade de comparações

Novos nomes (n)	Expressão geral	Com $Q = 1$
10	$10Q + 45$	55
20	$20Q + 190$	210
40	$40Q + 780$	820
80	$80Q + 3160$	3240
160	$160Q + 12720$	12880

3.2 Complexidade temporal e escopo do modelo

A expressão possui os termos nQ e n^2 . Logo, considerando n e Q como parâmetros, a quantidade de comparações cresce como $\Theta(nQ + n^2)$. Se Q permanece fixo enquanto n aumenta, o termo n^2 domina e a complexidade em função de n é $\Theta(n^2)$, portanto $O(n^2)$. Se Q também variar, a forma $\Theta(nQ + n^2)$ descreve o comportamento completo.

A unidade de custo adotada é uma comparação entre dois campos Nome, exatamente como definido no exercício. O custo interno de comparar caracteres dentro de strcmp e o custo de leitura/escrita em disco não são usados na função $C(n, Q)$, porque a atividade pede explicitamente a análise baseada na quantidade de comparações Nome×Nome.

3.3 Caso $Q = 0$ e orientação com Q cadastros iniciais

A análise geral mantém Q porque a orientação em sala solicitou considerar cadastros já existentes. Para a validação experimental foi fixado $Q=1$, pois esse caso representa um destino não vazio e reproduz os valores do exemplo de estatísticas do enunciado. Nesse caso, $C(n) = n(n+1)/2$. Se o destino começar vazio, isto é, $Q=0$, a mesma fórmula geral reduz-se a $C(n, 0) = n(n-1)/2$. Assim, ambos os cenários são casos particulares do mesmo modelo.

3.4 Crescimento quando n dobra

Para Q fixo, o termo dominante é n^2 . Portanto, ao dobrar n , espera-se que a quantidade de comparações se aproxime de quatro vezes o valor anterior. Com $Q=1$, as razões observadas foram 3,82; 3,90; 3,95 e 3,98. A aproximação de 4 melhora à medida que n cresce, pois os termos de menor ordem passam a representar uma parcela menor do total.

4. Demonstração por indução

Deseja-se provar, para todo $n \geq 1$:

$$C(n, Q) = nQ + n(n-1)/2$$

Caso base ($n = 1$). O primeiro novo bolsista é comparado com os Q registros existentes, portanto $C(1, Q) = Q$. Pela fórmula:

$$C(1, Q) = 1 \cdot Q + 1(1-1)/2 = Q$$

Hipótese de indução. Suponha que a expressão seja verdadeira para $n = k$:

$$C(k, Q) = kQ + k(k-1)/2$$

Passo indutivo. Ao inserir o registro $k+1$, já existem $Q+k$ registros no destino:

$$\begin{aligned} C(k+1, Q) &= C(k, Q) + (Q+k) \\ &= kQ + k(k-1)/2 + Q + k \\ &= (k+1)Q + k(k+1)/2 \\ &= (k+1)Q + (k+1)((k+1)-1)/2 \end{aligned}$$

Conclusão. A expressão é verdadeira para $n=1$ e, se é verdadeira para k , também é verdadeira para $k+1$. Logo, pelo princípio da indução matemática, $C(n, Q) = nQ + n(n-1)/2$ é válida para todo $n \geq 1$.

5. Metodologia experimental

A metodologia foi montada para testar diretamente a hipótese matemática. O objetivo não foi medir velocidade em segundos, mas verificar se o número real de comparações executado pelo programa coincide com $C(n, Q)$.

5.1 O que, como e por que foi medido

O QUE	COMO	POR QUÊ
Arquivo de entrada	Foi utilizado bolsistas-2025.csv, fornecido para a atividade.	Mantém o experimento ligado ao conjunto de dados real solicitado pelo professor.
Q inicial	O cadastro-base.csv foi criado com exatamente 1 registro ($Q=1$) e copiado novamente antes de cada experimento.	Mantém Q constante e impede que uma execução anterior altere a seguinte.
Entradas n	prepare_tests.py seleciona nomes únicos do CSV e gera arquivos com 10, 20, 40, 80 e 160 registros.	Esses são os tamanhos exigidos e a unicidade garante a hipótese usada na dedução.
Preparação dos testes	O script ignora o nome usado no cadastro-base e usa um conjunto de nomes já selecionados para impedir repetição.	Garante que cada busca percorra todo o destino, reproduzindo exatamente o caso teórico analisado.
Métrica observada	Cada comparação Nome×Nome incrementa comparacoes em C; ao final, nomes_inseridos e comparacoes são gravados em estatisticas.csv.	É uma medição determinística da operação pedida, portanto não sofre ruído de relógio, CPU ou sistema operacional.
Modelo teórico	plot.py calcula $C(n, Q) = nQ + n(n-1)/2$ para cada n e subtrai o valor observado.	A coluna diferença permite verificar objetivamente se a implementação segue a fórmula.
Gráfico	Pandas lê as estatísticas e Matplotlib desenha as curvas observada e teórica no mesmo eixo.	A sobreposição das curvas torna visual a concordância entre hipótese e experimento.

5.2 Ferramentas utilizadas

C e GCC foram usados para implementar e compilar o algoritmo solicitado. O módulo csv do Python foi usado para preparar conjuntos de teste reproduzíveis; Pandas foi usado para organizar e comparar as estatísticas; Matplotlib foi usado para gerar a representação gráfica. Essas ferramentas não substituem a análise matemática: servem para executar o algoritmo, preservar os dados e confrontar a previsão teórica com o comportamento observado.

5.3 Controles, limitações e validade do experimento

Os nomes dos experimentos são distintos e não aparecem no cadastro-base, o cabeçalho não é contado como registro e cada execução começa com uma cópia nova do mesmo arquivo base. Esses controles são necessários porque um nome repetido pode ser encontrado antes do fim da busca, reduzindo o número de comparações e quebrando a hipótese usada na fórmula. Como a variável medida é um contador lógico, o resultado é exatamente reproduzível para a mesma entrada. A limitação é que esta validação confirma a quantidade de comparações, não o tempo físico total do programa nem o custo interno de cada strcmp.

5.4 Validação funcional de redundância

Além dos testes de complexidade, foi executado um caso separado contendo um nome já presente no cadastro-base e um nome novo. O programa identificou o duplicado, exibiu a mensagem de erro, não inseriu a linha redundante, continuou a execução e inseriu o registro válido. Esse teste é separado porque a presença de duplicatas altera a quantidade de comparações, mas é necessária para comprovar o requisito funcional do programa.

6. Resultados experimentais

Hipótese experimental: para nomes distintos, com Q fixo, o contador observado deve ser exatamente igual a $C(n,Q)$; além disso, quando n dobra, a razão entre contagens sucessivas deve se aproximar de 4.

n	Q	Observado	Teórico	Diferença
10	1	55	55	0
20	1	210	210	0
40	1	820	820	0
80	1	3240	3240	0
160	1	12880	12880	0

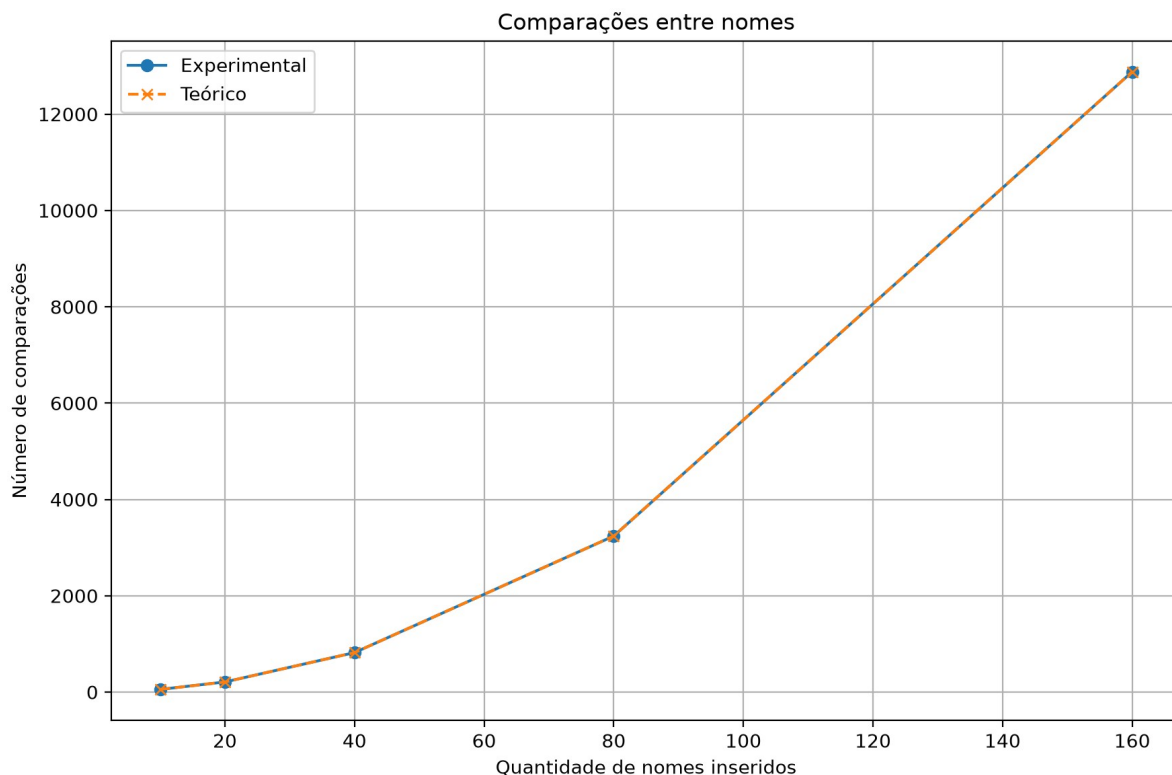


Figura 1 - Comparações teóricas e experimentais para $Q=1$.

7. Comparação entre hipótese e dados empíricos

Os cinco valores observados são idênticos aos previstos, portanto a diferença é zero em todos os casos. Isso confirma que o contador da implementação executa exatamente a sequência $Q, Q+1, \dots, Q+n-1$ prevista na dedução.

Quando n dobra, as razões são $210/55=3,82$; $820/210=3,90$; $3240/820=3,95$ e $12880/3240=3,98$. A aproximação progressiva de 4 é coerente com o termo dominante n^2 e fornece evidência empírica do comportamento quadrático para Q fixo.

A concordância exata não é resultado de acaso estatístico: como a variável observada é um contador lógico, a mesma entrada produz sempre o mesmo total. A indução estabelece a validade da fórmula para todo n ; os experimentos, por sua vez, verificam que o código implementado segue essa regra nos tamanhos solicitados. Assim, prova matemática e validação empírica se complementam.

Caso fossem introduzidos nomes repetidos, a busca poderia terminar antes de percorrer todo o destino e a contagem ficaria abaixo da fórmula usada neste experimento. Isso não contradiz o modelo: apenas viola a hipótese de nomes distintos e inexistentes que foi explicitamente adotada.

8. Conclusão

O algoritmo implementado transfere registros entre arquivos CSV e impede a inclusão de nomes redundantes por meio de busca sequencial. A solução foi instrumentada para contar exatamente as comparações Nome×Nome, porque essa é a operação definida como métrica da atividade.

Para n novos bolsistas e Q cadastros iniciais, a dedução resultou em $C(n,Q)=nQ+n(n-1)/2$. A expressão foi demonstrada por indução matemática. Nos experimentos com $Q=1$ e n igual a 10, 20, 40, 80 e 160, foram observadas 55, 210, 820, 3240 e 12880 comparações, exatamente os valores previstos; a diferença foi zero em todos os casos.

A taxa de crescimento se aproximou de quatro quando n dobrou, sustentando a classificação $\Theta(n^2)$ para Q fixo. Quando Q também é tratado como variável, a forma completa é $\Theta(nQ+n^2)$. Portanto, a implementação funcional, a dedução matemática, a prova por indução e os dados experimentais apontam para a mesma conclusão, fortalecendo a validade da hipótese proposta.

9. Repositório público

O repositório contém o código-fonte em C, o arquivo bolsistas-2025.csv, os conjuntos usados nos testes, os arquivos de saída, os CSVs de estatísticas, o gráfico e as instruções de compilação e execução:

<https://github.com/branchescunha/algorithm-analysis/tree/main/report-03-scholarship-registration-analysis>